1. Objetivo

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

Este trabalho tem como objetivo implementar o mecanismo de sincronização do embarque de passageiros de um vagão de metrô para a METROREC. É necessário garantir que o vagão só parta quando todos os passageiros que ele liberou estiverem sentados e que o embarque respeite a capacidade, usando exclusivamente as funções de **pthread_mutex_*** e **pthread_cond_***, com um único lock por estação, sem semáforos e sem busy wait, assumindo um vagão por carro e no máximo um carro na estação por vez.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar". Status: OK, Parcial ou Não feito.

Requisito do enunciado	**Status**	**Como testar / evidência**
1. Usar apenas **pthread_mutex_*** e **pthread_cond_***	OK	**sync_car.c** usa só **pthread_mutex_init/lock/unlock** e **pthread_cond_init/wait/signal/broadcast**, e a build passa com **-Werror**
2. Sem semáforos nem outras primitivas	OK	nenhuma chamada a **sem_*** no código, a sincronização fica só com mutex e condvar
3. No máximo um lock por **struct station**	OK	**sync_car.h** declara um único **pthread_mutex_t**, e **station_init** inicializa só ele
4. Premissa: um carro por vez, mesmo destino	OK	**sync_car-runner.c** cria uma thread de vagão por viagem, e a execução passou com essa premissa
5. Embarque simultâneo	OK	na execução a viagem com 35 assentos embarcou 35 passageiros sem asterisco, e o runner só chama **station_on_board** depois de contar cada retorno de **station_wait_for_car**
6. Sem busy-wait	OK	todas as esperas são **pthread_cond_wait** em **sync_car.c**, não existe laço de espera ativa
7. Makefile gera o executável **sync_car**	OK	**make clean && make** compila os dois objetos

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado. Compilar:

```
make clean
make
```

Executar:

```
./sync_car
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. [sync_car.h] -> [define struct station e os protótipos das funções]
2. [sync_car.c] -> [implementa station_init, station_wait_for_car, station_on_board, station_load_car]
3. [sync_car-runner.c] -> [teste fornecido pelo professor, com um bloco acrescentado no fim da main para o caso inválido: **station_load_car** com capacidade negativa deve retornar de imediato e não deixar nenhum passageiro embarcar. O restante do arquivo ficou intacto]
4. [Makefile] -> [build com -std=gnu11 -Wall -Wextra -Werror -pthread, gera o executável sync_car]

Decisões técnicas:

1. [Struct no header] -> [o runner faz struct station station na main] -> [logo precisa do tipo completo]
2. [Duas variáveis de condição, uma para passageiros e outra para o vagão] -> [cada grupo espera por um predicado diferente] -> [o **signal** nunca acorda a thread errada e o vagão não precisa de **broadcast**]
3. [**boarding** como contador de confirmações pendentes] -> [o vagão precisa esperar o embarque confirmado, e não só a reserva do assento] -> [o vagão só parte quando nenhum passageiro liberado ainda deve **station_on_board**]
4. [**car_present** e **free_seats** zerados antes do **unlock** no fim de **station_load_car**] -> [impede que um passageiro atrasado reserve assento de um vagão que já partiu] -> [o próximo vagão começa com estado limpo]

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Contadores de estado com um mutex e duas condvars	Fechei o contrato das quatro funções e o modelo de estado antes de escrever qualquer chamada a **pthread_***	não houve troca

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Primeiro build com **make clean && make**	OK		17:26	saída do **make clean && make** com os três comandos **gcc**
2	S1	Build final com as quatro funções implementadas	OK		2026-10-29 20:50:29-03:00	**make clean && make** de 20:50
3	S1	Executar o runner com 100 passageiros	OK		2026-10-29 20:50:29-03:00	saída de **./sync_car** de 20:50
4	S1	Rodar 30 execuções seguidas para procurar falha intermitente	OK		2026-10-29 21:45:50-03:00	evidencias.log
5	S1	Acrescentar ao runner um caso inválido, **station_load_car** com capacidade negativa, e rodar de novo	OK		2026-10-29 22:09:22-03:00	evidencias.log

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar.**

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.

- Validação final: teste passando / saída correta.

O único erro realmente relevante que tive, eu esqueci de tirar print, e depois não consegui reproduzir de novo pois não percebi todos os detalhes.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei". **Onde usei IA (2–3 linhas):**

Não foi utilizada inteligência artifial nem para geração de código nem para o preenchimento deste relatório.

O que validei manualmente e como: Compilei do zero com **make clean && make** e confirmei que o build passa com **-Werror**, sem aviso. Rodei o runner várias vezes e comparei o número de assentos oferecidos com o número de embarques. Escrevi um teste de estresse com o valgrind de 30 execuções para garantir que não falha. Escrevi e formatei este relatório completamente na mão em markdown. Utilizei a template do guia de relatórios disponibilizada no clasroom, reproduzi ela em markdown.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão: Comecei fechando o contrato das quatro funções e o modelo de estado, e isso me poupou tempo quando fui escrever as chamadas de **pthread_***. Tratar **boarding** como confirmações pendentes, em vez de guardar um alvo separado, deixou o estado menor e mais fácil de entender para mim. Com mais tempo eu rodaria o helgrind desde a primeira versão, em vez de deixar para o fim, e talvez criaria mais testes de limites e casos de borda para garantir que nada bai quebrar.

9. Checklist Final de Entrega

Item	**Confirmado?**
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico. Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Eu acho que a reflexão final já responde essa pergunta.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.